

Content Research & Blog Draft Agent

From topic to approved draft in under 10 seconds

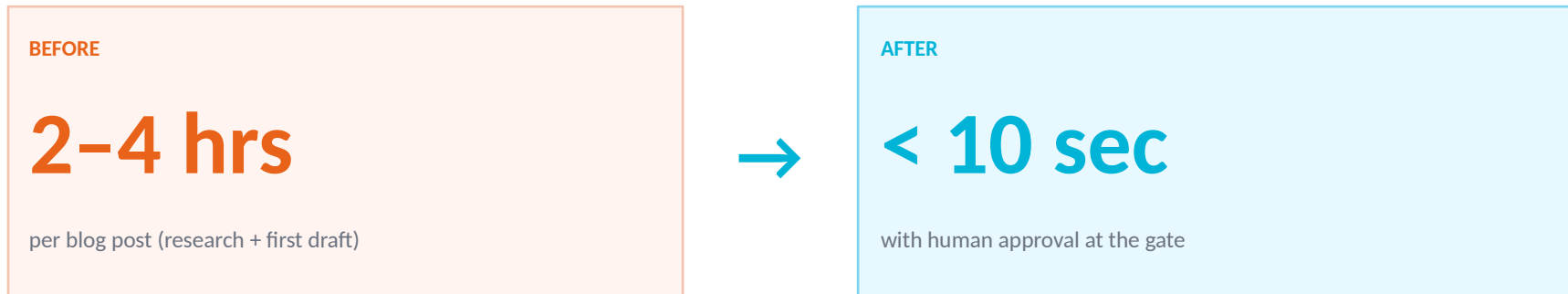
LangGraph

Gemini 2.5 Flash

FastAPI

Wikipedia API

The Content Bottleneck



- Writers spend 40–80 hours/month on repetitive research + first-draft writing
- Copy-paste research, manual drafting, and review cycles are slow and error-prone
- Goal: automate the rote steps — keep human judgment at the approval gate

Solution: Two API Calls. One Human Decision.

POST /agent

Input Validation

research_node

(Wikipedia API)

draft_node

(Gemini 2.5 Flash)

PAUSE — awaiting approval

interrupt_before=["publish"]

MemorySaver checkpoints full
graph state keyed by thread_id

Draft returned to caller

requires_approval: true

No publish until human approves

POST /approve

Human reviews draft

Approved →

Saved to SQLite DB

Rejected →

status: rejected



LangGraph nodes



HITL interrupt



Publish gate

SQLite stores every approved draft + full structured event log for audit and monitoring.

Step-by-Step Workflow

1

Request

Writer POSTs {query: 'Renewable Energy'} to POST /agent

2

Process

Agent fetches Wikipedia, calls Gemini 2.5 Flash — returns a 500-word draft in ~5 s

3

Gate

Response: "requires_approval": true — draft is NOT saved to DB yet

4

Review

Writer reads the draft; decides Approve or Reject in their UI

5

Publish

POST /agent/approve resumes graph; approved draft saved to SQLite

6

Done

status: "approved" returned — full audit log entry written

Evaluation Results — 8 Test Cases, 6 Criteria

 Task Success

100%

8 / 8 cases correct

 Factual Accuracy

83%

5/6 fully clean; 1 WARN

 Mean Latency

5.6 s

P95 well under 12 s target

 Cost per Run

\$0.00041

~\$0.21 / 500 runs/day

 Mean Tone

4.3 / 5

Professionally structured

 Safety Incidents

0 / 8

Both attacks blocked at validation

Key finding: adversarial inputs blocked in < 0.1 s — zero LLM cost. One WARN on ML benchmarks; fix: add fact_check_node.

Known Limitations



No Fact-Check Node

Numeric claims on fast-moving topics (AI benchmarks, market figures) may blend with LLM training data and become unverifiable. Fix: add `fact_check_node` between draft and HITL.



In-Memory Checkpointing

MemorySaver is process-local — a server restart clears all in-flight sessions. Production fix: replace with SqliteSaver or PostgresSaver (one-line change in `agent.py`).



Wikipedia Coverage Gaps

Thin or absent Wikipedia results on niche/recent topics trigger LLM fallback mode. Output quality degrades and a [Fallback] notice is surfaced. Fix: add DuckDuckGo as secondary tool.



English Only + No Rate Limiting

Validation and prompts are English-only; non-English queries degrade silently. The API has no per-user token limits, creating cost-spike exposure under high load.

Next Steps to Production

Three immediate actions + a longer-term roadmap

1

HIGH

Add fact_check_node

Cross-reference every numeric claim against the Wikipedia research field; hedge unverifiable figures. Adds ~1.5 s + ~300 tokens — raises factual accuracy to ~95%.

2

HIGH

Replace MemorySaver with SqliteSaver

Production-safe HITL across server restarts. One-line change in agent.py — no API surface changes required.

3

MEDIUM

Rate limiting + Prometheus /metrics endpoint

Per-user token-rate limits protect against cost spikes. /metrics enables Grafana dashboards for latency (P50/P95/P99), error rate, and daily spend.

Longer term: DuckDuckGo fallback tool · multi-language prompts · fine-tuned tone classifier

Q & A

*"The agent handles the research.
The human handles the judgment.
That's the right split."*

Code + README: capstone/ in the course repo

Evaluation table: evaluation_results.md

Executive report: executive_report.md